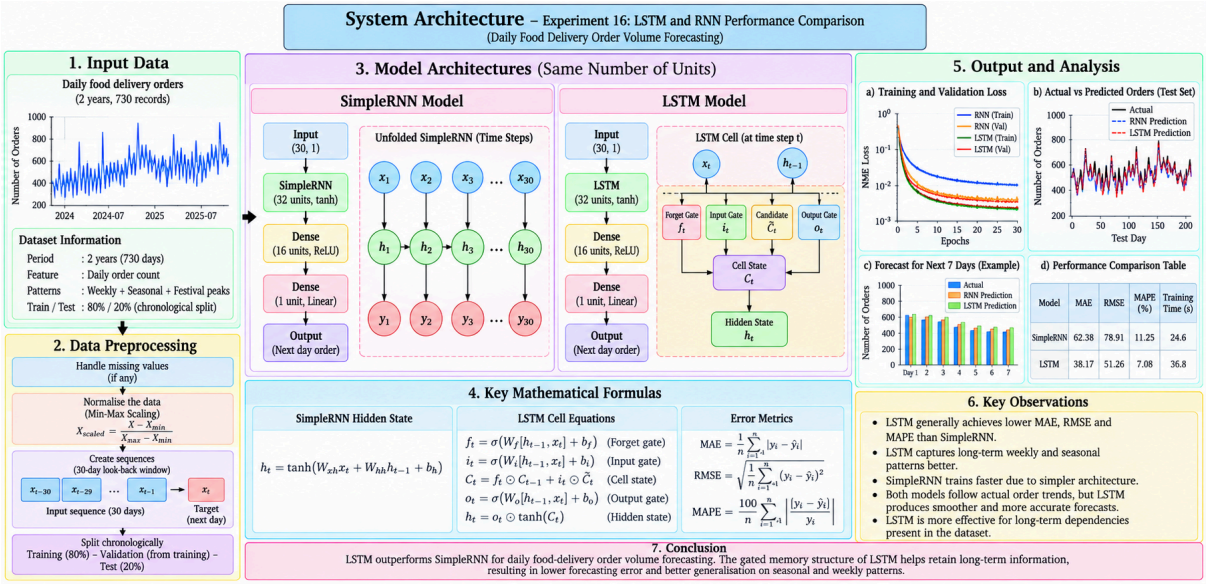
Experiment 16: LSTM and RNN Performance Comparison

1. Aim

To compare the performance of Simple Recurrent Neural Network (SimpleRNN) and Long Short-Term Memory (LSTM) models for daily food-delivery order-volume forecasting. To prepare time-series sequences using a 30-day look-back window, train both models with the same number of hidden units, compare training loss, validation loss, and training time, and evaluate forecasting performance using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE). Actual and predicted order volumes are visualized using Matplotlib.



2. Mathematical Formulas

1. Sliding Window

$$\boxed{X_t = [x_{t-30}, x_{t-29}, \ldots, x_{t-1}]}$$

$$\boxed{Y_t = x_t}$$

The previous 30 days are used to predict the next day's order volume.

2. SimpleRNN Hidden State

$$\boxed{h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)}$$

3. LSTM Forget Gate

$$\boxed{f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)}$$

4. LSTM Input Gate

$$\boxed{i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)}$$

5. LSTM Cell State

$$\boxed{C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t}$$

6. LSTM Output Gate

$$\boxed{o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)}$$

$$\boxed{h_t = o_t \odot \tanh(C_t)}$$

7. Mean Absolute Error (MAE)

$$\boxed{MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|}$$

8. Root Mean Squared Error (RMSE)

$$\boxed{RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}}$$

CODE

```
# Experiment 16: LSTM and RNN Performance Comparison

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import time

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

np.random.seed(42)
tf.random.set_seed(42)

# ========================================
# 1. GENERATE TWO YEARS OF ORDER DATA
# ========================================

days = 365 * 2

dates = pd.date_range(
    start="2024-01-01",
    periods=days,
    freq="D"
)

t = np.arange(days)

# Base daily orders
base_orders = 500

# Weekly pattern
weekly_pattern = 100 * np.sin(2 * np.pi * t / 7)

# Seasonal pattern
seasonal_pattern = 80 * np.sin(2 * np.pi * t / 365)

# Festival and special-day demand
festival_pattern = np.zeros(days)
festival_days = [50, 120, 200, 300, 400, 500, 600, 700]

for day in festival_days:
    festival_pattern[day] = 250

# Random variations
noise = np.random.normal(0, 35, days)

# Final order volume
orders = (
    base_orders
    + weekly_pattern
    + seasonal_pattern
    + festival_pattern
    + noise
)

orders = np.maximum(orders, 50)

df = pd.DataFrame({
    "Date": dates,
    "Orders": orders
})
```

```
print("Food Delivery Dataset:")
print(df.head())

print("\nTotal Records:", len(df))

# ========================================
# 2. VISUALIZE DAILY ORDER VOLUME
# ========================================

plt.figure(figsize=(12, 5))

plt.plot(df["Date"], df["Orders"])

plt.xlabel("Date")
plt.ylabel("Number of Orders")
plt.title("Daily Food Delivery Order Volume")
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 3. NORMALIZE THE DATA
# ========================================

values = df["Orders"].values.reshape(-1, 1)

# Chronological 80:20 split
split_index = int(len(values) * 0.80)

train_values = values[:split_index]

scaler = MinMaxScaler()

# Fit only on training data
scaler.fit(train_values)

scaled_values = scaler.transform(values)

print("\nNormalized Data Sample:")
print(scaled_values[:5])

# ========================================
# 4. CREATE 30-DAY SLIDING WINDOWS
# ========================================

window_size = 30

X, y, target_indices = [], [], []

for i in range(window_size, len(scaled_values)):

    X.append(scaled_values[i-window_size:i])
    y.append(scaled_values[i])
    target_indices.append(i)

X = np.array(X, dtype="float32")
y = np.array(y, dtype="float32")
target_indices = np.array(target_indices)

# Chronological split based on target date
train_mask = target_indices < split_index
test_mask = target_indices >= split_index

X_train_full = X[train_mask]
y_train_full = y[train_mask]
```

```
X_test = X[test_mask]
y_test = y[test_mask]

# Create chronological validation set
val_index = int(len(X_train_full) * 0.85)

X_train = X_train_full[:val_index]
y_train = y_train_full[:val_index]

X_val = X_train_full[val_index:]
y_val = y_train_full[val_index:]

print("\nSequence Dataset Shapes:")
print("Training:", X_train.shape)
print("Validation:", X_val.shape)
print("Testing:", X_test.shape)

# ========================================
# 5. BUILD SIMPLE RNN MODEL
# ========================================

def build_rnn():

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(30, 1)),

        tf.keras.layers.SimpleRNN(
            32,
            activation="tanh"
        ),

        tf.keras.layers.Dense(16, activation="relu"),

        tf.keras.layers.Dense(1)
    ])

    model.compile(
        optimizer="adam",
        loss="mse"
    )

    return model

# ========================================
# 6. BUILD LSTM MODEL
# ========================================

def build_lstm():

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(30, 1)),

        tf.keras.layers.LSTM(
            32,
            activation="tanh"
        ),

        tf.keras.layers.Dense(16, activation="relu"),

        tf.keras.layers.Dense(1)
    ])

    model.compile(
```

```
        optimizer="adam",
        loss="mse"
    )

    return model

# ========================================
# 7. TRAIN SIMPLE RNN
# ========================================

tf.keras.backend.clear_session()
tf.random.set_seed(42)

rnn_model = build_rnn()

start_time = time.perf_counter()

rnn_history = rnn_model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=30,
    batch_size=32,
    shuffle=False,
    verbose=1
)

rnn_time = time.perf_counter() - start_time

print("\nSimpleRNN Training Time:",
      round(rnn_time, 2), "seconds")

# ========================================
# 8. TRAIN LSTM
# ========================================

tf.keras.backend.clear_session()
tf.random.set_seed(42)

lstm_model = build_lstm()

start_time = time.perf_counter()

lstm_history = lstm_model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=30,
    batch_size=32,
    shuffle=False,
    verbose=1
)

lstm_time = time.perf_counter() - start_time

print("\nLSTM Training Time:",
      round(lstm_time, 2), "seconds")

# ========================================
# 9. COMPARE TRAINING AND VALIDATION LOSS
# ========================================

plt.figure(figsize=(10, 5))
```

```
plt.plot(
    rnn_history.history["loss"],
    label="SimpleRNN Training Loss"
)

plt.plot(
    rnn_history.history["val_loss"],
    label="SimpleRNN Validation Loss"
)

plt.plot(
    lstm_history.history["loss"],
    label="LSTM Training Loss"
)

plt.plot(
    lstm_history.history["val_loss"],
    label="LSTM Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("SimpleRNN vs LSTM: Training and Validation Loss")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 10. MAKE TEST PREDICTIONS
# ========================================

rnn_pred_scaled = rnn_model.predict(
    X_test, verbose=0
)

lstm_pred_scaled = lstm_model.predict(
    X_test, verbose=0
)

# Convert predictions to original order scale
actual = scaler.inverse_transform(y_test)

rnn_pred = scaler.inverse_transform(
    rnn_pred_scaled
)

lstm_pred = scaler.inverse_transform(
    lstm_pred_scaled
)

# ========================================
# 11. CALCULATE MAE, RMSE AND MAPE
# ========================================

def calculate_metrics(actual, predicted):

    mae = mean_absolute_error(
        actual, predicted
    )

    rmse = np.sqrt(
        mean_squared_error(actual, predicted)
    )
```

```
    mape = np.mean(
        np.abs((actual - predicted) / actual)
    ) * 100

    return mae, rmse, mape

rnn_mae, rnn_rmse, rnn_mape = calculate_metrics(
    actual, rnn_pred
)

lstm_mae, lstm_rmse, lstm_mape = calculate_metrics(
    actual, lstm_pred
)

print("\nSIMPLE RNN RESULTS")
print("MAE :", round(rnn_mae, 2))
print("RMSE:", round(rnn_rmse, 2))
print("MAPE:", round(rnn_mape, 2), "%")

print("\nLSTM RESULTS")
print("MAE :", round(lstm_mae, 2))
print("RMSE:", round(lstm_rmse, 2))
print("MAPE:", round(lstm_mape, 2), "%")

# ========================================
# 12. PLOT ACTUAL VS PREDICTED ORDERS
# ========================================

plt.figure(figsize=(12, 5))

plt.plot(
    actual[:100],
    label="Actual Orders"
)

plt.plot(
    rnn_pred[:100],
    label="SimpleRNN Prediction"
)

plt.plot(
    lstm_pred[:100],
    label="LSTM Prediction"
)

plt.xlabel("Test Day")
plt.ylabel("Number of Orders")
plt.title("Actual vs Predicted Daily Order Volume")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 13. CREATE COMPARISON TABLE
# ========================================

comparison = pd.DataFrame({
    "Model": ["SimpleRNN", "LSTM"],
    "MAE": [rnn_mae, lstm_mae],
    "RMSE": [rnn_rmse, lstm_rmse],
    "MAPE (%)": [rnn_mape, lstm_mape],
    "Training Time (seconds)": [rnn_time, lstm_time]
})
```

```
print("\nMODEL PERFORMANCE COMPARISON")
print(comparison.round(3).to_string(index=False))

# ========================================
# 14. FIND THE BETTER MODEL
# ========================================

if lstm_rmse < rnn_rmse:
    print("\nLSTM achieved lower RMSE.")
else:
    print("\nSimpleRNN achieved lower or equal RMSE.")

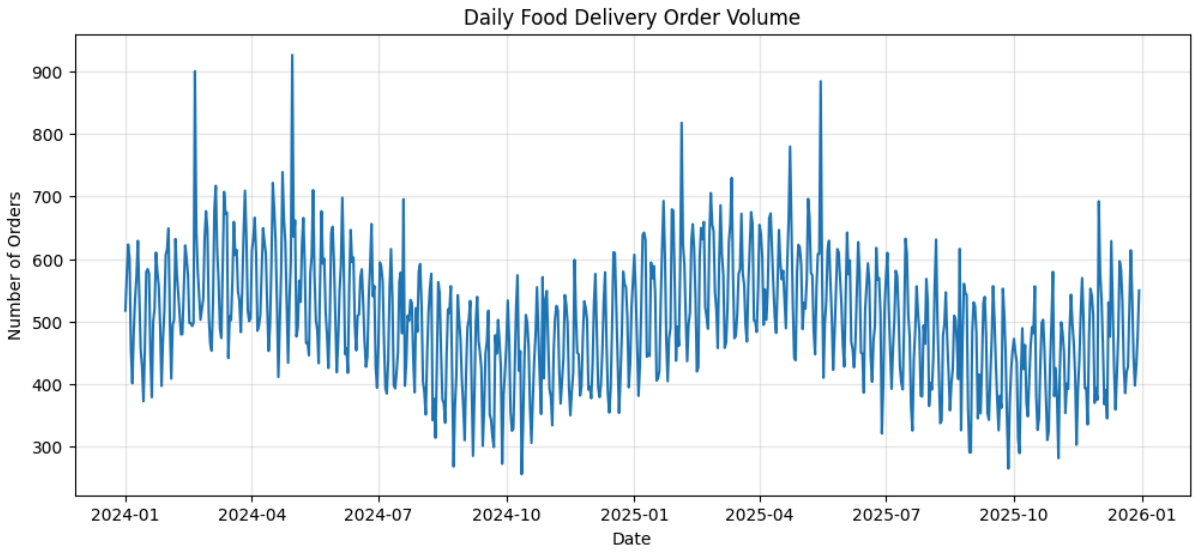
print("\nExperiment Completed Successfully!")
```

OUTPUT

Food Delivery Dataset:

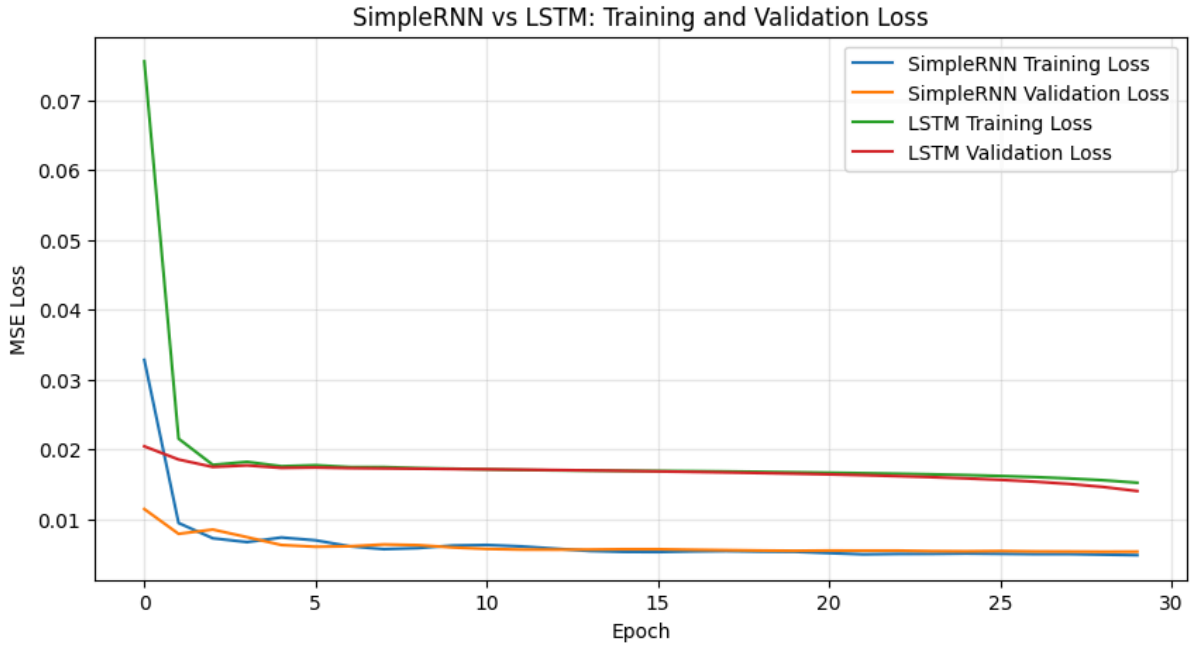
	Date	Orders
0	2024-01-01	517.384995
1	2024-01-02	574.720966
2	2024-01-03	622.915619
3	2024-01-04	600.823992
4	2024-01-05	453.920452

Total Records: 730



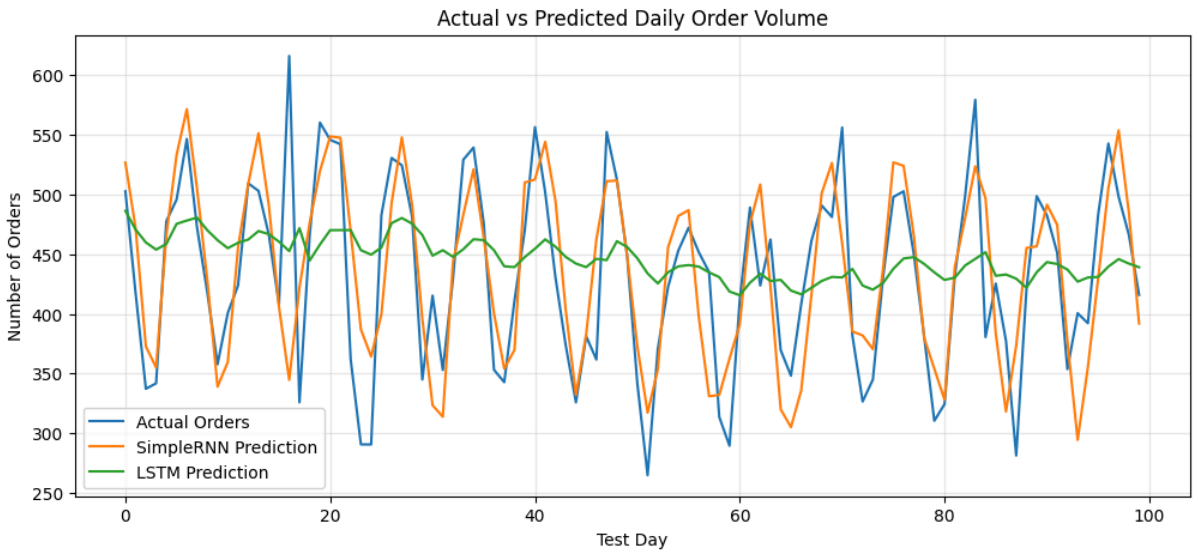
Normalized Data Sample:
[[0.3903]
 [0.4758]
 [0.5476]
 [0.5147]
 [0.2956]]

Sequence Dataset Shapes:
Training: (470, 30, 1)
Validation: (84, 30, 1)
Testing: (146, 30, 1)



SIMPLE RNN RESULTS
MAE : 39.57
RMSE: 53.99
MAPE: 9.26 %

LSTM RESULTS
MAE : 63.39
RMSE: 77.08
MAPE: 15.4 %




```
MODEL PERFORMANCE COMPARISON
    Model    MAE  RMSE  MAPE (%)  Training Time (seconds)
SimpleRNN 39.573 53.99     9.259                    8.700
     LSTM 63.392 77.08    15.398                    7.884

SimpleRNN achieved lower or equal RMSE.
```

5. RESULT

SimpleRNN and LSTM models were implemented using TensorFlow and Keras to forecast daily food-delivery order volumes. Two years of time-series data were generated, normalized, and converted into supervised sequences using a 30-day sliding window. Both models were trained with 32 hidden units and evaluated using MAE, RMSE, and MAPE. Training and validation loss curves, actual-versus-predicted graphs, and a performance comparison table were generated. The experiment demonstrates the differences between SimpleRNN and LSTM in learning sequential patterns and handling long-term dependencies in time-series forecasting.